Phase 1 Revision: Questions and Answers

Foundations and Setup, Days 1 to 14. Every concept as a question, with the commands to go with it.

Remember: Cover the answer, say it out loud, then check. If you cannot explain it in your own words, you have not learned it yet (Learn, Build, Push, Explain). The command cheat sheet is at the end.

1. Roadmap and Setup (Day 1)

Q1. What is the loop this roadmap follows?

Learn, Build, Push, Explain. Like cooking: you watch the dish (learn), cook it yourself even if it comes out ugly (build), save proof of what you did to GitHub (push), and tell a friend how you made it (explain). Repeat it every day.

Q2. What does a Playwright automation engineer do?

Writes code that opens a real browser, clicks buttons, types text and checks the result, in seconds. At a real company you:

- Write automated tests for the product
- Run them whenever developers change something
- Find bugs and report them clearly
- Keep the tests healthy as the product changes

Q3. Why do we install VS Code, Node.js and Git?

Tool	What it is	Why we need it
VS Code	A notebook for code	Where you write and open files and the terminal
Node.js (LTS)	The engine that runs JavaScript	Playwright needs it. LTS = Long Term Support, the stable version
Git	A save-point system for files	Keeps history, lets you go back, and pushes work to GitHub

Q4. How do I check Node is installed?

```
node --version
```

You should see something like `v22.x.x`. If you see "command not found", close VS Code, reopen it and try again.

2. The Terminal (Days 2 and 3)

Q5. What is a terminal and why do we use it?

A text window where you give your computer written instructions instead of clicking. Git, Node and Playwright are all run from here, so you use it every day on the job. In VS Code: Terminal, then New Terminal.

Q6. What is the difference between an absolute and a relative path?

	Absolute path	Relative path
Meaning	Full address from the drive. Works from anywhere	Address from where you are standing. Depends on your location
Example	`D:\qspiders\bible\playwright\Playwright-Learning\Phase1`	`.\Phase1` or `Phase1`
Analogy	House 5, Lake Road, Bengaluru, India	The second house on your left

Special symbols: `.` means "here" and `..` means "one level up".

Q7. What are the basic navigation and file commands?

Command	Meaning	Example
`pwd`	Print working directory: where am I?	`pwd`
`ls`	List files and folders here	`ls`
`cd`	Change directory (go into a folder)	`cd Phase1`
`cd ..`	Go up one folder	`cd ..`
`mkdir`	Make a folder	`mkdir notes`
`New-Item`	Create a file (`touch` on Mac/Linux)	`New-Item day02.md -ItemType File`
`Set-Content`	Write text into a file (replaces what was there)	`Set-Content day02.md "text"`
`Add-Content`	Add a line to the end of a file	`Add-Content day02.md "more"`
`cat`	Show what is inside a file	`cat day02.md`
`ls -R`	List everything, including subfolders	`ls -R`
`ls -Force`	Show hidden items such as `.git`	`ls -Force`
`cls` / `clear`	Wipe the screen. Nothing is deleted	`cls`

Q8. How do I build this tree using only the terminal? Phase1/ with notes/, projects/, practice/

```
pwd                                  # should end with Playwright-Learning
cd Phase1
mkdir notes
mkdir projects
mkdir practice
ls
cd notes
New-Item day02.md -ItemType File
Set-Content day02.md "Today I learned pwd, ls, cd, mkdir and cat."
cat day02.md
cd ..
New-Item projects\readme.md -ItemType File     # relative path: no need to enter projects
New-Item practice\readme.md -ItemType File
ls -R                                # see the whole tree
```

Q9. How do I copy, move, rename and delete?

```
mkdir fruits
New-Item apple.md -ItemType File
mv apple.md fruits\                          # MOVE into a folder
cp fruits\apple.md fruits\apple-backup.md   # COPY (original stays)
mv fruits\apple-backup.md fruits\apple2.md  # RENAME (same command, new name)
rm fruits\apple2.md                         # DELETE one file
mkdir test
rm test -Recurse                            # delete a folder with contents
```

`mv` does two jobs: if the destination is a folder it moves the file there, if it is a new name it renames the file.

Q10. Why is `rm` dangerous and what are the safety rules?

`rm` does not use the Recycle Bin. A deleted file is gone, and Git cannot bring back files you never committed. Rules:

- Run `pwd` and `ls` before every `rm`
- Delete one specific name at a time. Never use `*` (everything) with `rm`
- Only practice deleting inside your `practice` folder
- `-Recurse` deletes without asking in PowerShell, so check twice

Q11. What are the three terminal time-savers?

- **Tab-completion:** type `cd Ph` and press Tab. It finishes the word and prevents typos (like "Pkaywright")
- **Command history:** Up arrow brings back the previous command. `Get-History` lists this session's commands
- **Running a program:** type its name and press Enter. Your own code runs with `node filename.js`

```
Set-Content hello.js 'console.log("Hello from Node!")'
node hello.js                        # prints: Hello from Node!
```

Q12. I am on Mac or Linux. What are the equivalents?

Windows PowerShell	Mac / Linux
`New-Item f.md -ItemType File`	`touch f.md`
`Set-Content f.md "text"`	`echo "text" > f.md`
`Add-Content f.md "text"`	`echo "text" >> f.md`
`rm test -Recurse`	`rm -r test`
`ls -Force`	`ls -a`
`cls`	`clear`

3. Git and GitHub (Days 4, 5 and 6)

Q13. What is the difference between Git and GitHub?

	Git	GitHub
What it is	A program (version control system)	A website and online service
Where it lives	Your computer	The internet
Needs internet?	No	Yes
Main job	Save versions locally	Store, share and collaborate
Can it work alone?	Yes	No, it needs Git
Analogy	Save points in a video game (or the camera)	Cloud album (or the online photo album)

Alternatives to GitHub: GitLab and Bitbucket. Git works with all of them.

Q14. What problem does Git solve?

The mess of `report_final.docx`, `report_final_v2.docx`, `report_FINAL_really_final.docx`. Git saves snapshots, remembers what changed, when and by whom, and lets you return to any earlier snapshot if something breaks.

Q15. What are the three places a file travels through?

Working folder, then staging area (the box), then repository (history). The box lets you commit only the files that belong together.

```
Working folder --git add--> Staging area (box) --git commit--> Repository (history)
```

File states: **Untracked, Modified, Staged, Committed.**

Q16. What is a commit?

A snapshot of your project plus who made it, when, your message and a unique ID. It is stored on your computer in the hidden `.git` folder until you push.

Q17. What is the one-time setup (once per computer)?

```
git --version                                  # check Git is installed
git config --global user.name "Your Name"
git config --global user.email "your-github-email@example.com"
git config --global --list                     # confirm name and email
```

`--global` means "remember this for every project on this computer".

Q18. How do I start a project and connect it to GitHub (once per project)?

```
git init                                       # NEVER run again inside an existing repo
git add .
git commit -m "Day 001: Welcome & Setup"
git branch -M main                             # name the main branch "main"
git remote add origin https://github.com/YOURUSERNAME/Playwright-Learning.git
git push -u origin main                        # -u links local main to GitHub main
```

`origin` is just the nickname for the GitHub address. After `-u`, a plain `git push` is enough. Create the empty repo on GitHub first, and do not tick "Add README" if you already have files locally.

Q19. What is my daily Git routine?

```
git status              # where do things stand?
git add .               # choose what to save (put in the box)
git diff --staged       # check the box before sealing it
git commit -m "Day 00X: topic"
git push                # send to GitHub
git log --oneline       # confirm the commit is there
```

Remember: Edit, status, diff, add, diff --staged, commit, log --oneline, status.

Q20. What does `git status` do and how do I read it?

It reports only, and changes nothing. Run it before `git add` and after every step.

What you see	What it means	Next step
Untracked files	New files Git is not watching yet (red)	`git add .`
Changes not staged for commit	Known files you edited, not in the box yet	`git add .`
Changes to be committed	Files in the box, ready to seal (green)	`git commit -m "..."`
Your branch is ahead of 'origin/main' by 1 commit	Saved locally, not pushed	`git push`
nothing to commit, working tree clean	Working folder matches the last commit, box is empty	Nothing to do

Tip: `git status -s` gives a short format.

Q21. What are the different `git diff` commands?

Command	Compares
`git diff`	Working folder vs staging area (changes NOT yet staged)
`git diff --staged`	Staging area vs last commit (what WILL go into the next commit)
`git diff HEAD`	Working folder vs last commit (everything)
`git diff --stat`	A short summary of changed files

Lines starting with `-` were removed and `+` were added. `git status` says WHICH files changed, `git diff` says WHAT changed. If `git diff` shows nothing but you changed files, they are probably already staged: use `--staged`. It also shows nothing for brand-new untracked files.

Q22. How do I read my history?

```
git log --oneline       # one line per commit, newest first
git log --oneline -5    # last 5 commits
git log --oneline --graph --all
git show                # latest commit: who, when, exact change
```

HEAD is a pointer to where you are now, usually the latest commit of your current branch. If the screen shows `:` or `(END)`, you are inside a pager: press **q** to quit.

Q23. What is `.gitignore` and when does it fail?

A file listing patterns Git should not track. Why: protect secrets (`.env`), skip generated files (`node_modules/`), skip OS junk (`.DS_Store`), keep the repo small. Add it in the first commit.

```
node_modules/
.env
*.log
.DS_Store
test-results/
playwright-report/
```

Patterns: `*.log` (any log file), `folder/` (a folder), `!file` (exception), `#` (comment). **It only affects untracked files.** For a file already committed, stop tracking it with:

```
git rm --cached secrets.txt
git commit -m "Stop tracking secrets.txt"
```

If a secret was already pushed, it stays in history: rotate (change) the secret.

Q24. What makes a good commit message?

- Imperative mood: "Add", not "Added"
- Subject about 50 characters, no full stop
- Say what and why, not how
- One logical change per commit (atomic)
- Test: "If applied, this commit will ___"

Bad: "fixed bugs and added stuff". Good: "Fix crash when username is empty" and a separate "Add email field to signup form".

Q25. What is a branch and why use one?

A movable pointer to a commit: instant and cheap. You work on a branch without touching the stable `main` until the work is ready.

```
git switch -c feature/notes      # create a branch and switch to it
git branch                       # list branches, * marks the current one
git add .
git commit -m "Add notes on branches"
git switch main                  # go back to main
git merge feature/notes          # bring the branch into main
git push
git branch -d feature/notes      # delete the merged branch
```

Merge rule: switch to the target branch first, then merge the other branch into it. **Fast-forward** means `main` had no new commits, so Git just moves the pointer.

Q26. How do I undo mistakes before committing?

```
git restore notes.md             # discard UNSAVED edits (permanent!)
git restore --staged notes.md    # take it out of the box, keep the edit
```

`restore` only works on uncommitted changes, and discarding cannot be undone.

Q27. How do I fix a wrong remote URL?

```
git remote -v                    # show what origin points to
git remote set-url origin https://github.com/YOURUSERNAME/Playwright-Learning.git
```

Q28. What is inside the hidden `.git` folder?

```
ls -Force      # .git appears (mode starts with d--h)
ls .git
```

It is your whole repository and history. Look only. Never edit or delete anything in there.

Q29. Which Git errors did I meet and how do I fix them?

Error or symptom	Cause	Fix
No configured push destination	Git does not know where GitHub is	`git remote add origin <url>`
remote origin already exists	The nickname `origin` is already used	`git remote set-url origin <url>`
Push goes to a wrong address	Typo in the URL	`git remote -v`, then `set-url`
Screen shows `:` or `(END)`	You are inside a pager	Press `q`
Author identity unknown	Name and email not set	`git config --global user.name` and `user.email`
Push rejected	GitHub has a commit you lack (e.g. a README made on GitHub)	`git pull origin main --rebase`, then `git push`
not a git repository	Wrong folder	`pwd`, then `cd` to the repo root

4. Web Basics and Chrome DevTools (Day 7)

Q30. What are the client and the server?

The **client** asks (your browser, a mobile app). The **server** is the computer that answers by storing files and data and sending them back. Flow: the client sends a **request**, the server sends a **response**. Analogy: you are the customer, the server is the kitchen, HTTP is the waiter.

Q31. What are the parts of a URL?

```
https://www.example.com:443/blog/post?id=7&lang=en#comments
```

Part	Example	Meaning
Protocol	`https`	Rules for talking (s = secure)
Subdomain	`www`	Optional prefix
Domain	`example.com`	Human-readable name of the server
Port	`:443`	Door on the server. Usually hidden (https = 443, http = 80)
Path	`/blog/post`	Which page or resource
Query	`?id=7&lang=en`	Extra key=value data sent to the server
Fragment	`#comments`	Jump to a section. Handled by the browser only

Q32. What is in an HTTP request and response, and what are the methods?

Request = method, URL, headers, optional body. Response = status code, headers, body (HTML, JSON, image).

Method	Use
`GET`	Fetch data
`POST`	Send or create data
`PUT` / `PATCH`	Update data
`DELETE`	Remove data

Q33. What do the status codes mean?

Code	Meaning
`200`	OK
`301` / `302`	Redirect
`400`	Bad request
`401` / `403`	Not logged in / not allowed
`404`	Not found: the server works, but nothing exists at that URL
`500`	Server error: the request was fine, the server hit a problem

Remember: 2xx = success, 3xx = redirect, 4xx = your fault (client), 5xx = server's fault.

Q34. What do HTML, CSS and JavaScript each do?

Language	Role	Analogy
HTML	Structure and content	Skeleton
CSS	Style and layout	Clothes
JavaScript	Behavior and interaction	Muscles

Playwright finds elements using HTML (tags, ids, classes, text), and JS is what changes the page after it loads.

Q35. What is the DOM?

The browser's live, in-memory map of the page, built from the HTML. Every heading, paragraph and button is a node, nested like folders. JavaScript changes the DOM and the screen updates, while the original HTML file stays untouched. **HTML is the recipe on paper, the DOM is the dish on the table that you can still add salt to.** Playwright interacts with the DOM.

```
Page
 \-- Body
      |-- Heading: "Welcome"
      |-- Paragraph: "Hello there"
      \-- Button: "Click me"
```

Q36. What does a browser do when I enter a URL?

- 1. Finds the server and sends a request
- 2. Server sends back HTML
- 3. Browser reads the HTML and requests more files (CSS, JS, images)
- 4. Builds the DOM
- 5. Applies CSS and runs JS
- 6. Renders the page on screen

Q37. What are the three DevTools tabs and what are they for?

Open with **F12**, **Ctrl + Shift + I**, or right-click and Inspect.

Tab	Shows	Practice
Elements	The live DOM and CSS. You can edit temporarily	Click the select arrow, click an element, note its tag, id, class. Double-click text to edit, refresh to reset
Network	Every request: method, status, type, size, time	Refresh with the tab open, click a request, check Headers and Response. Filters: Doc, JS, CSS, Img, Fetch/XHR
Console	Errors and logs. Lets you run JavaScript	Commands below

```
document.title
document.querySelector('textarea')    // quick check: does this locator find my element?
console.log('Hello DevTools')
document.title = "Changed"
```

5. Software Testing Basics (Day 8)

Q38. What is software testing and what are its goals?

Checking that a product does what it should and does not do what it should not, before real users find the problems.

- Find bugs early (cheaper to fix early)
- Confirm the software meets requirements
- Build confidence in quality
- Reduce risk (money loss, security, bad experience)

Testing shows that bugs exist, never that there are none. A **bug/defect** is a mismatch between expected and actual behavior.

Q39. What is the difference between verification and validation?

Verification: "Are we building the product right?" (does it follow the spec). **Validation:** "Are we building the right product?" (does it meet user needs).

Q40. What is the difference between SDLC and STLC?

	SDLC	STLC
Covers	The whole journey of building software	Only the testing activities
Phases	Requirements, Design, Development, Testing, Deployment, Maintenance	Requirement analysis, Test planning, Test case design, Environment setup, Execution, Closure
Relationship	The big picture	A part inside the SDLC

Memory trick: SDLC builds the house, STLC inspects the house. Models: Waterfall (step by step), Agile (short sprints), V-model (each dev phase paired with a test phase).

Q41. Manual vs automation testing: what is the difference?

	Manual	Automation
Who runs it	A person	Scripts and tools
Best for	New features, usability, exploring	Regression, repetitive, large-scale
Upfront cost	Low	High
Cost over time	Grows every run	Shrinks every run
Weakness	Slow, inconsistent	Brittle, blind to the unexpected

Q42. How do I decide what to automate?

Ask three questions: **Will I run this many times? Is the behavior stable? Is the answer clearly pass or fail?** If all three are yes, automate it. If not, test by hand.

Common mistakes: automating everything, automating too early, or never automating.

Q43. What does an automation engineer or SDET do?

- Builds test automation frameworks
- Writes automated UI, API and sometimes performance tests
- Plugs tests into CI/CD pipelines (Jenkins, GitHub Actions)
- Analyzes failures and finds root causes, fixes flaky tests
- Works with developers and manual testers on what to automate

Automation engineer uses tools to automate tests. **SDET** (Software Development Engineer in Test) is more coding-heavy and builds the frameworks and tools. Titles vary by company. Tools to recognize: Selenium, Playwright, Cypress (web), Appium (mobile), Postman, REST Assured (API), JUnit, TestNG, pytest (frameworks).

6. Test Design (Day 9)

Q44. Test scenario vs test case?

	Test scenario	Test case
Level	High-level idea	Detailed and specific
Answers	What to test?	How to test it?
Contains	One line	ID, preconditions, data, steps, expected result, priority
Count	Few	Many per scenario

Scenario: "Verify login with invalid credentials." Test case: open the login page, enter `testuser@mail.com` and `Wrong@999`, click Login. Expected: "Invalid credentials" and the user stays on the login page. The scenario is the chapter title, the test cases are the pages.

Q45. Positive vs negative testing?

Positive: valid input, confirm it works (correct email + password). **Negative:** invalid or unexpected input, confirm it fails safely with a clear error, no crash, no leak (wrong password, blank fields, `' OR '1'='1`). Both are needed. Blank fields on a login page are a negative test.

Q46. What is equivalence partitioning (EP)?

Divide all inputs into groups where the system behaves the same, then test **one value per group**. You cannot test every input, so EP cuts tests while keeping coverage. Example: password length 8 to 16.

Partition	Range	Valid?	Test value
Too short	0 to 7	Invalid	4
Valid	8 to 16	Valid	12
Too long	17+	Invalid	20

Q47. What is boundary value analysis (BVA)?

Bugs live at the edges (off-by-one mistakes such as < instead of <=). Test just below, on, and just above each boundary. A random value like 47 passes whether the code is right or wrong, but 0, 1, 100 and 101 expose the mistake.

Field	Valid range	BVA values
Password length	8 to 16	7, 8, 9, 15, 16, 17
Age	18 to 60	17, 18, 19, 59, 60, 61
Number	1 to 100	0, 1, 2, 99, 100, 101

Q48. How do EP and BVA work together?

EP first (split into valid and invalid groups), BVA next (test the edges). Valid groups give positive tests, invalid groups give negative tests. For age 18 to 60: EP picks 10, 35, 70. BVA picks 17, 18, 19, 59, 60, 61. Negative extras: letters and blank.

Q49. What columns should a test case table have?

ID, Preconditions, Test data, Steps, Expected result, Priority. Good test cases have one purpose each, are specific, repeatable by someone else, and have a clear expected result. Weak: "Check login works." Strong: "Enter testuser@mail.com and Wrong@999, click Login, expect an Invalid credentials error and stay on the login page."

7. Bug Life Cycle and Bug Reports (Day 10)

Q50. What is the bug life cycle?

```
New -> Assigned -> Open -> Fixed -> Ready for retest -> Retest -> Verified -> Closed
If the bug is still there after retest: Reopened (back to the developer)
```

Side paths where the bug is not fixed:

Status	Meaning
Rejected / Invalid	Not actually a bug (working as designed)
Duplicate	Already reported
Deferred	Real bug, fix pushed to a later release
Cannot reproduce	Developer cannot make it happen, needs more info
Won't fix	Known issue, decided not to fix

Q51. What does a good bug report contain?

It lets a developer reproduce and fix the bug without asking you anything.

Field	What to write
Title	What broke and where. Bad: "Search broken". Good: "Search returns 500 error when keyword contains &"
Environment	Browser + version, OS, device, build, test environment, user role
Steps	Numbered, exact, short, with test data. Say how often it happens (5 out of 5)
Expected	What should happen, based on the requirement
Actual	What really happened
Severity / Priority	See next question
Evidence	Screenshot, recording, console errors, Network tab details (URL, method, status, response), logs

Q52. Severity vs priority?

	Severity	Priority
Meaning	How badly the bug damages the product	How urgently it should be fixed
Set by	Usually the tester	Usually product owner / manager
Based on	Technical impact	Business need and timing
Levels	Critical, Major, Minor, Trivial	High, Medium, Low (or P1, P2, P3)

Q53. Give an example of each severity and priority combination.

Combination	Example
High severity, high priority	Payment fails for every user
High severity, low priority	App crashes, but only on a rarely used obsolete browser
Low severity, high priority	Company name misspelled on the home page
Low severity, low priority	Slight misalignment on an admin-only screen

Q54. What are the tips for writing great bug reports?

- One bug per report
- Be factual, not emotional: "button does not respond"
- Reproduce it twice before reporting, and search for duplicates first
- Describe the product's behavior, do not blame the developer
- Isolate it: does it happen in another browser or with other data?

Q55. What happens when a retest fails? Name two statuses where a bug is not fixed.

It is **Reopened** and goes back to the developer. Not fixed: Rejected, Duplicate, Deferred, Won't fix, Cannot reproduce.

8. Test Strategy (Day 11)

Q56. What is the test pyramid and why that shape?

```
      /\
     /UI\          few  - slow, brittle, expensive
    /----\
   / API  \        some - moderate speed, stable
  /--------\
 /   Unit   \      many - fast, cheap, precise
```

Cost and flakiness rise as you go up, feedback speed falls, and failures at the top only say "something broke somewhere". **Rule: test each thing at the lowest level that can catch the bug.** Example: test all 50 discount rules as unit tests, check the `/cart` total at API level, and keep one UI happy-path test.

Anti-patterns: **ice cream cone** (mostly manual/UI tests), **hourglass** (thin API layer). The **testing trophy** is an alternative with integration tests as the biggest layer.

Q57. Unit vs API vs UI tests?

	Unit	API / Integration	UI / E2E
Tests	One function in isolation	Endpoints, services, DB	Full user flow in the browser
Speed	Milliseconds	100 ms to seconds	Seconds to minutes
Stability	Very stable	Fairly stable	Often flaky
Failure tells you	The exact function	The endpoint	Vague, needs digging
Tools	JUnit, pytest, Jest	Postman, REST Assured, Supertest	Playwright, Selenium, Cypress

Q58. Smoke vs sanity vs regression?

	Smoke	Sanity	Regression
Question	Is the build even testable?	Does this specific fix work?	Did we break anything that worked?
Scope	Broad and shallow	Narrow and deep	Broad and deep
When	Every new build or deploy	After a bug fix	Before release, nightly
Outcome	Fail = reject the build	Fail = back to dev	Gate for release

Memory aid: **Smoke** = turn it on, does smoke come out? **Sanity** = is the fix sane? **Regression** = re-run the old stuff. Definitions vary between companies, so state yours.

Q59. What is the difference between retesting and regression testing?

Retesting re-runs the specific failed test to confirm the fix. **Regression** checks that other existing functionality still works. After a fix: sanity and retest first, then regression around it.

Q60. What should be automated and what should not?

Automate	Keep manual or exploratory
Repetitive, stable regression tests	Exploratory testing (needs curiosity)
Smoke tests in CI/CD	Usability and UX feel
Data-driven tests with many inputs	Features still changing
High-risk paths: login, checkout, payment	One-off or rarely run tests
Cross-browser and device matrix	Unreliable environments or data
Performance, load, API contract tests	Subjective visual checks (tools help partly)

Q61. What is the decision checklist before automating?

- Will it run many times?
- Is the feature stable?
- Is the expected result clear and deterministic?
- Is the business risk high?
- Is it cheaper to automate than to run manually over its lifetime?
- Can it be done at a lower layer of the pyramid instead?

If most answers are yes, automate it. Automation frees humans to do the work only humans do well.

Q62. What is a flaky test and why does it matter?

A test that sometimes passes and sometimes fails with the same code. Example: the test clicks Login and immediately checks the dashboard. If the server takes 3 seconds, the check runs too early and fails, although the app is fine.

Causes: timing (not waiting), slow or unstable environment, shared or dirty data, test order dependency, external services, random or time-based values. Harm: you stop trusting failures, real bugs get ignored, CI slows down. UI tests are flakier than unit tests because a browser, network, server and database are all involved. **A flaky test is a smoke alarm that goes off randomly.**

Q63. Where does each test type run in a pipeline?

Stage	Tests
On commit or PR	Unit tests, static analysis, fast API tests
After deploy to test env	Smoke suite
After a bug fix	Sanity and confirmation test
Nightly or pre-release	Full regression, exploratory, usability, performance
After production deploy	Smoke test in production

9. Markdown, README and Notes (Day 12)

Q64. What is Markdown and what is the syntax?

A plain-text format that a renderer (GitHub, VS Code, Notion) turns into headings, bold text and lists. Files end in `.md`.

```
# Heading 1 (page title, use once)
## Heading 2 (main sections)
### Heading 3
**bold**   *italic*   ~~strike~~   `inline code`
- Bullet item
  - Nested (indent 2 spaces)
1. First step
- [ ] Todo     - [x] Done
[Link text](https://example.com)
![Alt text](images/screenshot.png)
> Blockquote
---
| Tool    | Layer |
|---------|-------|
| Jest    | Unit  |
```

Code blocks use triple backticks with the language name after the opening ones (for example `python` or `bash`), and triple backticks again to close. Always put a space after `#`.

Q65. What are the common Markdown mistakes?

- No blank line before a list or code block, so it does not render
- Forgetting the closing triple backticks, which breaks the rest of the page
- Using headings just to make text bigger
- Link text like "click here" instead of a meaningful description
- Skipping heading levels (H1 to H3)

Q66. What goes in a good README?

The README is the front door: a reader should know in 30 seconds what the project is and whether it is for them.

Section	What goes in it
Title + one line	What it does, in plain words
Screenshot or badges	Shows it works
Features	3 to 6 bullets
Getting started	Prerequisites, install, run. Copy-pasteable
Usage	Real examples with expected output
Running tests	The exact command
Contributing and License	How to help, which license

Rules: lead with the what and why, make setup copy-pasteable, test your own instructions, show do not tell, keep it current, link out to `/docs` for depth.

Q67. How do I write notes that teach?

- **Own words:** copying means you have not understood yet
- **Start with the why:** the problem it solves
- **Always add an example:** a concept without an example is a slogan
- **Compare look-alikes** in a table (smoke vs sanity vs regression)
- **Structure for scanning:** headings, bullets, bold only for key terms
- **Add a memory hook:** an analogy or one-liner
- **End with self-test questions** (active recall beats rereading)
- **Record your confusion** in a Confused section and resolve it later

Q68. What is my note template?

```
# YYYY-MM-DD: Topic
## 1. Learned      what I can do or understand now (own words, with example)
## 2. Confused     what is still unclear (resolve one every day)
## 3. Built        what I actually wrote or ran (code and result)
## 4. Explained    teach it to a friend in 3 to 5 sentences
## Self-test       3 questions to answer tomorrow without looking
```

10. Thinking Like a Programmer (Day 13)

Q69. Why learn to break a problem into steps?

Computers do exactly what you tell them, nothing more. Big problems feel impossible and small steps feel easy. Planning before coding is cheaper than fixing code. Every Playwright test is a list of steps: open page, type, click, check.

Q70. What is IPO (Input, Process, Output)?

Every program takes **input**, does a **process**, and gives **output**. Ask: What do I get? What do I do with it? What do I give back?

Problem	Input	Process	Output
Total bill	Price, quantity	Multiply, add tax	Total amount
Login	Username, password	Match against stored data, count attempts	Access or error
Playwright test	Steps, test data	Run in browser, compare actual to expected	Pass or fail

Q71. What is pseudocode? Show an example.

Plain-English steps that look like code but are not a real language. Write it first, then convert to real code. Common words: READ, PRINT, IF / ELSE, FOR / WHILE, SET.

```
START
READ age
IF age >= 18
    PRINT "Eligible to vote"
ELSE
    PRINT "Not eligible"
END IF
END
```

Q72. What do the flowchart shapes mean?

Shape	Meaning
Oval	Start / End
Rectangle	A step or action
Diamond	A decision (Yes / No)
Parallelogram	Input / Output
Arrow	Direction of flow

Q73. How do all of these fit together?

```
Problem -> Break into steps -> Identify IPO -> Write pseudocode -> Draw flowchart -> Code
```

Algorithm = a clear list of steps to solve a problem. Decomposition = splitting a big problem into small steps. Example (ATM): insert card, enter PIN, choose Withdraw, enter amount, check balance is enough, give cash, return card.

11. Project: Test Plan for saucedemo.com (Day 14)

Q74. What is a test plan and what does it contain?

A short document the team agrees on before testing: what will be tested, how, and when to stop.

Part	What it says
Objective	Why we are testing
Scope	In scope AND out of scope (out of scope sets expectations)
Risk ranking	Checkout and login = High, burger menu = Low. Test the money path first
Approach	Manual, exploratory, negative, regression, smoke, automation later
Environment and test data	URL, browsers, users, products
Entry and exit criteria	Entry = when you can start. Exit = when you can stop
Defect management	Severity, priority, lifecycle
Risks and deliverables	What could go wrong, what you will hand over

Q75. What are the entry and exit criteria for this project?

Entry: site reachable, plan and cases reviewed, test users available. **Exit:** all 20 cases executed, all P1 cases pass or are logged as bugs, no open Critical bug without a decision, every bug documented.

Q76. What are the saucedemo facts I need?

Item	Value
URL	https://www.saucedemo.com
Password for all users	`secret_sauce`
Users	`standard_user` (normal), `locked_out_user` (blocked), `problem_user` (built-in faults), `performance_glitch_user` (slow login)
Tax	8 percent
Prices	Backpack 29.99, Bike Light 9.99, Bolt T-Shirt 15.99, Fleece Jacket 49.99, Onesie 7.99, Red T-Shirt 15.99

Q77. How do I calculate an expected total? (Backpack + Bike Light)

Item total 29.99 + 9.99 = **39.98**. Tax 39.98 x 0.08 = 3.1984, rounded **3.20**. Total **43.18**. For the Fleece Jacket alone: item 49.99, tax 3.9992 rounded 4.00, total 53.99. Always calculate expected results yourself.

Q78. What were the 3 bugs and how were they rated?

ID	Bug (problem_user)	Severity	Priority
BUG-001	All products show the same image	Major	P2
BUG-002	Sorting has no effect	Major	P2
BUG-003	Last Name field fails at checkout, blocking the purchase	Critical	P1

`problem_user` is built with intentional faults, but we treat them as real defects, as we would in a real product. Always reproduce a bug yourself and attach screenshots before filing.

Q79. How many test cases are enough?

There is no magic number. It depends on **risk** (more cases for high-risk areas), **techniques** (EP and BVA tell you how many), **time**, and **diminishing returns** (the 10th similar test finds almost nothing new). Add a case only if you can finish: "This catches ___ that no other test does."

Q80. What must I do after writing the project?

```
# 1. Run all 20 cases, update the Status column, add screenshots
# 2. Push everything
git status
git add .
git commit -m "Add saucedemo test plan, 20 test cases and 3 bug reports"
git push
```

Terminal (PowerShell)

Command	What it does
`pwd`	Where am I?
`ls` / `ls -R` / `ls -Force`	List here / list all subfolders / show hidden items
`cd folder` / `cd ..`	Go into a folder / up one level
`mkdir name`	Create a folder
`New-Item f.md -ItemType File`	Create a file (`touch f.md` on Mac/Linux)
`Set-Content f.md "text"`	Write text into a file (replaces)
`Add-Content f.md "text"`	Append a line to a file
`cat f.md`	Show file contents
`cp a b` / `mv a b`	Copy / move or rename
`rm f` / `rm folder -Recurse`	Delete a file / delete a folder (permanent)
`cls` / `clear`	Clear the screen
`Get-History`	List commands typed in this session
`node file.js` / `node --version`	Run a JS file / check Node
Tab, Up arrow	Auto-complete a name / previous command

Git

Command	What it does
`git --version`	Check Git is installed
`git config --global user.name "Name"`	Set your name (once)
`git config --global user.email "mail"`	Set your email (once)
`git config --global --list`	Show your settings
`git init`	Turn a folder into a Git project (once)
`git status` / `git status -s`	Where do things stand? / short version
`git add .`	Put all changes in the staging box
`git diff` / `git diff --staged` / `git diff HEAD`	Unstaged changes / staged changes / everything vs last commit
`git commit -m "msg"`	Seal the box into a save point
`git log --oneline` / `-5` / `--graph`	Compact history / last 5 / branch shape
`git show`	Details of the latest commit
`git branch -M main`	Name the main branch `main`
`git remote add origin <url>`	Connect to GitHub (once)
`git remote -v` / `git remote set-url origin <url>`	Show the address / fix the address
`git push -u origin main` / `git push`	First push / every later push
`git branch`	List branches (* = current)
`git switch -c name` / `git switch name`	Create and switch / switch
`git merge name`	Merge a branch into the current one
`git branch -d name`	Delete a merged branch
`git restore f`	Discard unsaved edits (permanent)
`git restore --staged f`	Unstage a file, keep the edit
`git rm --cached f`	Stop tracking a file already committed
`git pull origin main --rebase`	Bring GitHub's commits in before pushing

Daily routine

```
git status
git add .
git diff --staged
git commit -m "Day 00X: topic"
git push
git log --oneline
```

Browser Console (DevTools)

```
document.title
document.querySelector('selector')
console.log('text')
```

Playwright preview (Phase 2)

```
npm init playwright@latest
npx playwright test
npx playwright test --headed
npx playwright show-report
```

Remember: Phase 1 map to Playwright: terminal runs the tests, Node executes them, the DOM is what locators point at, the Network tab leads to API testing, test cases become test scripts, bug reports are what you write when a test fails, and Git is where it all lives.